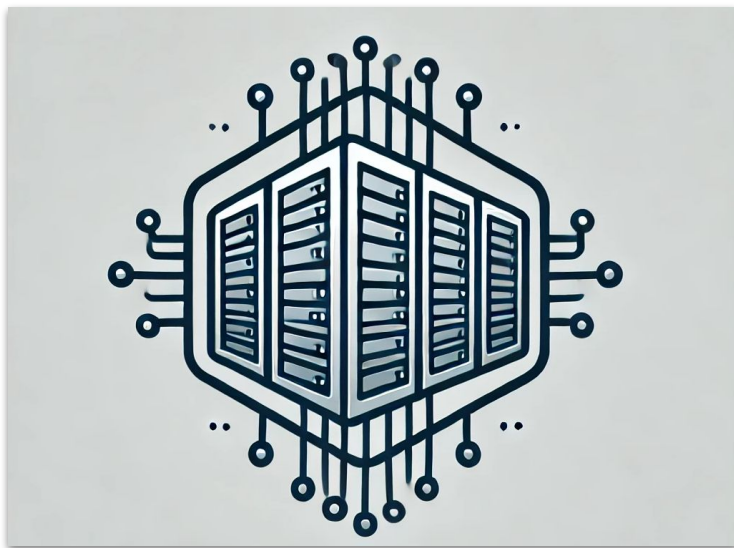


CSC 746: High Performance Computing

Optimizing use of cache

10 Sep 2026



Hello world

Last time:

- CP1 discussion
- Cache optimizations: copy, block+copy
- Hello CP2

Today:

- Review: CP2; copy, block+copy optimization
- Another optimization: in-memory layout

Next time:

- Deeper look at previous work on optimizations for multi-level cache systems

Upcoming dates/deadlines:

CP#2 due: 9/21/2026

CP#2 presentations: 9/22/2026

Need 2 volunteers to give a
CP#2 presentation

Volunteer by: Thu 9/17/2026
Else The Cat will Choose!

Memory Accesses (review)

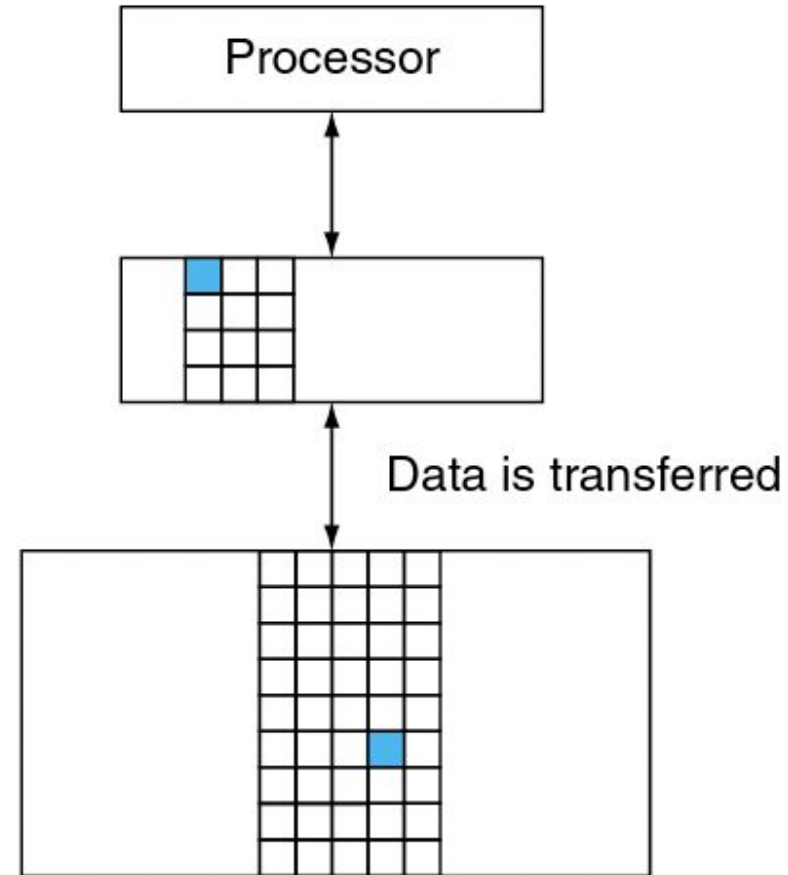
Memory Hierarchy, ctd.

When a program requests data...

Hit time: the time required to access a level of memory in the hierarchy, including time to determine if the access is a hit or miss

Miss penalty: the time required to fetch a block from a lower level of memory

- Latency+bandwidth+add'l overhead



Direct-mapped caches, multi-word blocks

Configuration:

- $N_b = 256$ entries (blocks)
- 16 words/block

Byte offset bits needed: 2

→ because have 2^2 bytes per word

Block offset bits needed: 4

→ because have 2^4 words per block

Block index bits needed: 8

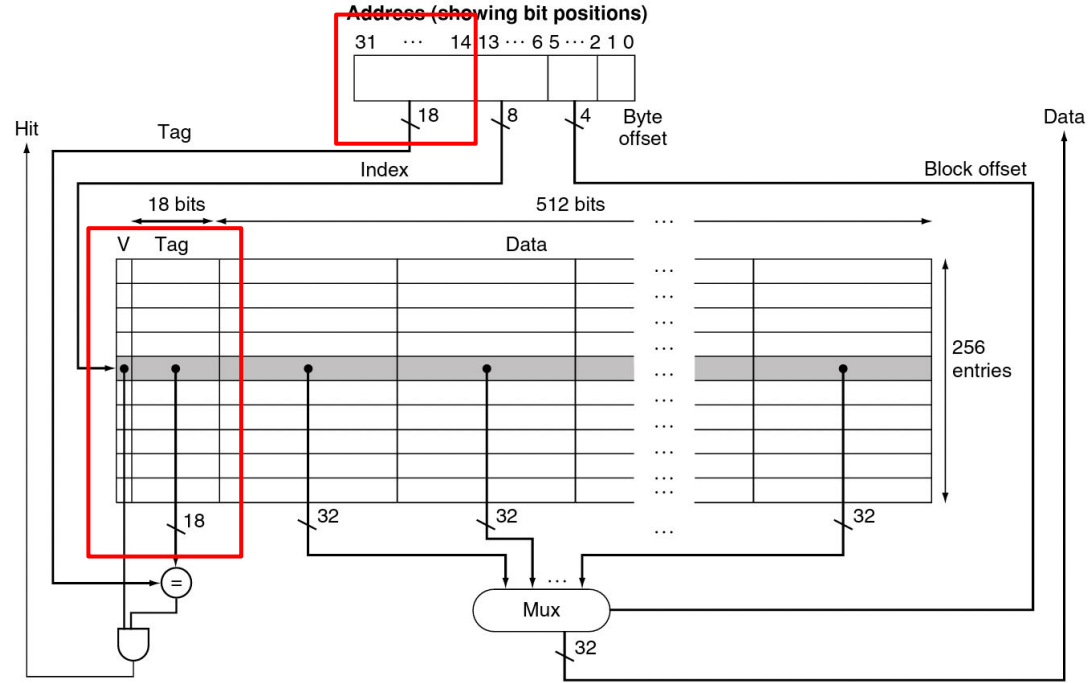
→ because have 2^8 blocks in cache

Tag bits needed: 18

→ have 32 bits of memory address

→ and 8+4+2 bits used for block index, block offset, and byte offset

→ $32 - (8 + 4 + 2) = 18$



Increasing the Cache Block Size

As the block size *increases*:

- The miss rate may go up because the number of blocks that fit into a cache will *decrease*
- The cost of a *miss penalty* will increase because we are having to access more data from lower in the memory hierarchy

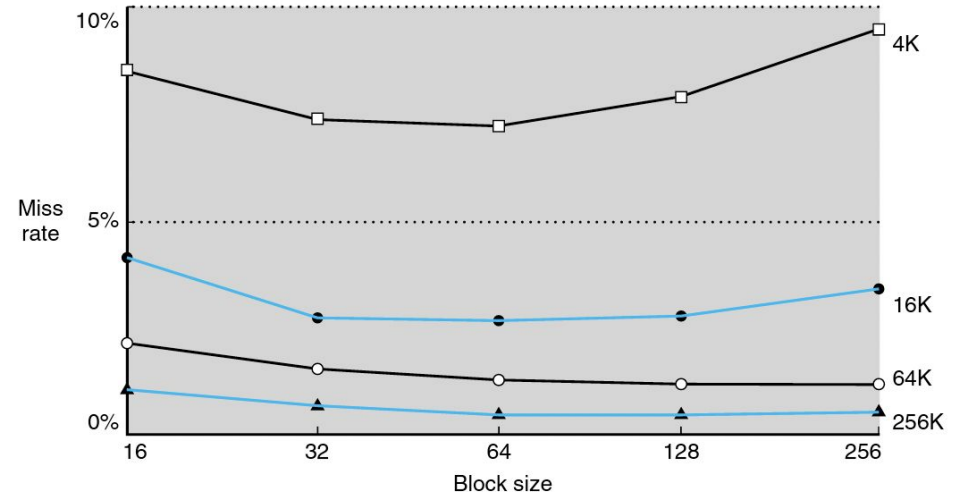


FIGURE 5.11 Miss rate versus block size. Note that the miss rate actually goes up if the block size is too large relative to the cache size. Each line represents a cache of different size. (This figure is independent of associativity, discussed soon.) Unfortunately, SPEC CPU2000 traces would take too long if block size were included, so this data is based on SPEC92.

DGEMM and Optimizations (CP#2)

Why Matrix Multiplication?

An important kernel that appears in many problems

- Used in many linear algebra algorithms
- Is expensive: $O(N^3)$ for $N \times N$ matrices (best known method $\sim O(N^{2.37})$)
- Is in the Top500 benchmark (LU factorization)
- Key to training time in DL (eg CNNs)

Optimization ideas can be used in other problems

The most-studied algorithm in High Performance Computing

How to Improve Use of Cache?

(Hardware) reduce the hit time: small, simple L1 cache

(Hardware) increase cache bandwidth through pipelined, multi-banked cache

(Hardware) reduce miss penalty (buffer management and priority)

(Software) compiler optimizations and user code: loop reordering, copy optimization, blocking and tiling, in-memory data layout strategies

(Hardware and software) reduce miss penalty rate via parallelism, prefetch

Vector-matrix multiplication: $y = y + A * x$

Assume:

x, y are vectors of length n

A is an $n \times n$ matrix

for $i=1:n$ // 1-based indexing

for $j=1:n$

$$y[i] = y[i] + A[i,j] * x[j]$$

How many memory references?

$2n$: we will access each $Y[i]$ twice ($r+w$)

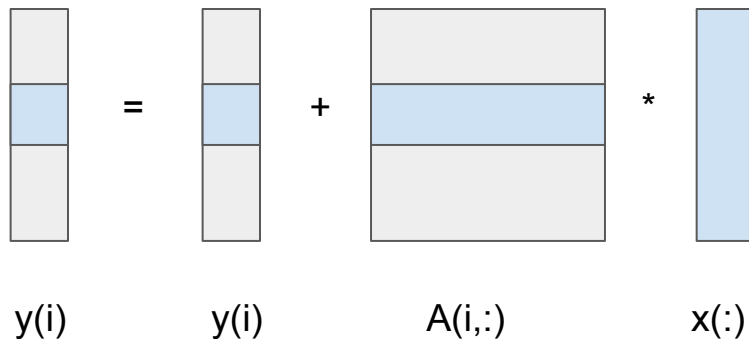
n^2 : we will access all of A once (r only)

n^2 : we will access all of X n times (r only)

$$m = 2n + 2n^2$$

f = number of arithmetic operations = $2n^2$

$$CI = f/m \approx n^2 / (n + n^2) < 1.0 \text{ (not so good)}$$



Copy optimization

Copy optimization:

- Copy a subset of a larger problem into a smaller buffer
- Perform operations on the smaller buffer
- The hope: the smaller buffer lives in cache
- The intention: repeated reuse of values in cache lowers AMAT
- Relies on: spatial, temporal locality of accesses

Vector-matrix mult improvement: copy optimization

Assume:

x, y are vectors of length n

A is an nxn matrix

Key idea: copy from main memory into small working set buffer that hopefully fits into cache

{read/copy x[1:n] into fast memory}
{read/copy y[1:n] into fast memory}

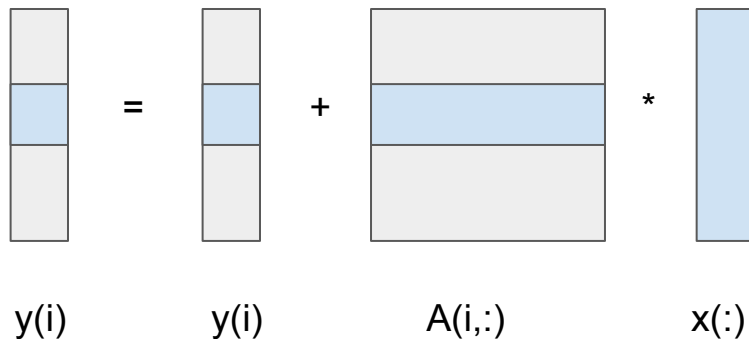
for i=1:n

{copy row i of A into fast memory}

for j=1:n

$$y[i] = y[i] + A[i,j] * x[j]$$

{write/copy y[1:n] back to slow memory}



VMM: Copy optimization can reduce number of slow memory accesses

With copy optimization:

- Access all of A once (r only)
 - Inside loop, A accesses are from fast mem
- Access y twice (r+w)
- Access x once (r only) (was 2x with original approach)

Number of slow memory accesses:

“Schoolbook” approach: $\underline{2n + 2n^2}$

Copy optimization: $\underline{3n + n^2}$

f = number of arithmetic operations = $2n^2$

CI = $f/m \approx 2$

- Better than $CI < 1$ from before, but still relatively low computational intensity
- Vector-matrix multiplication performance dominated by memory accesses

Assumption: size of fast memory large enough to accommodate $3N$ values

Original approach: how many mem refs?

$2n$: we will access each $Y[i]$ twice (r+w)

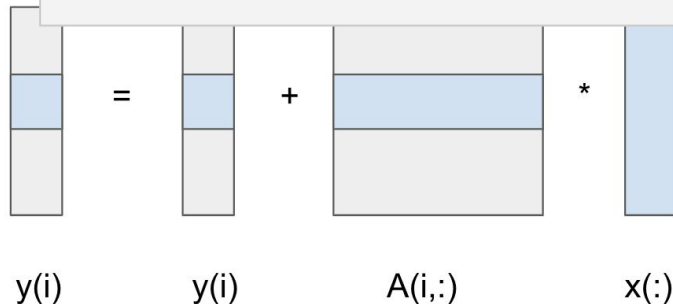
n^2 : we will access all of A once (r only)

n^2 : we will access all of X n times (r only)

$$m = 2n + 2n^2$$

f = number of arithmetic operations = $2n^2$

CI = $f/m \approx n^2/(n+2n^2) < 1.0$ (not so good)



Matrix multiply (basic version)

// implements $C = C + A * B$

// A, B, C are nxn matrices

for i=1:n // 1-based indexing

for j=1:n

for k=1:n

$C[i,j] += A[i,k] * B[k,j]$

For an $N \times N$ matrix:

MMUL is N^2 vector dot products

Each dot product requires $2N$ FLOPs

$f = 2N^3$ = number of Floating Point Operations

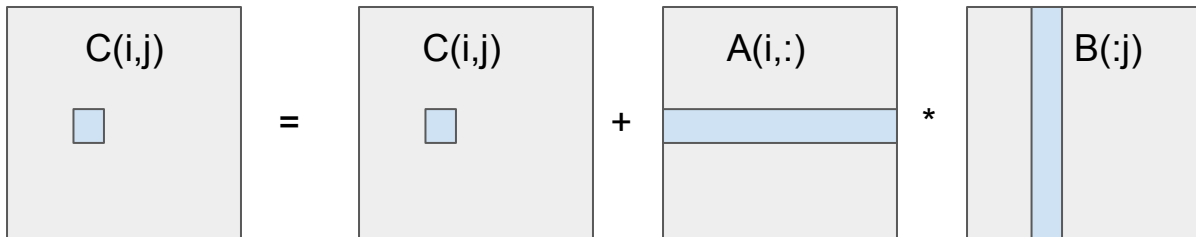
$Cl = f/m$

m = number of (slow) memory accesses

- N^3 accessing all of A, N times
- N^3 accessing all of B, N times
- $2N^2$ accessing all of C 2 times

$m = 2N^3 + 2N^2$

$Cl = f/m = (2N^3) / (2N^3 + 2N^2) < 1.0$ (not so good)



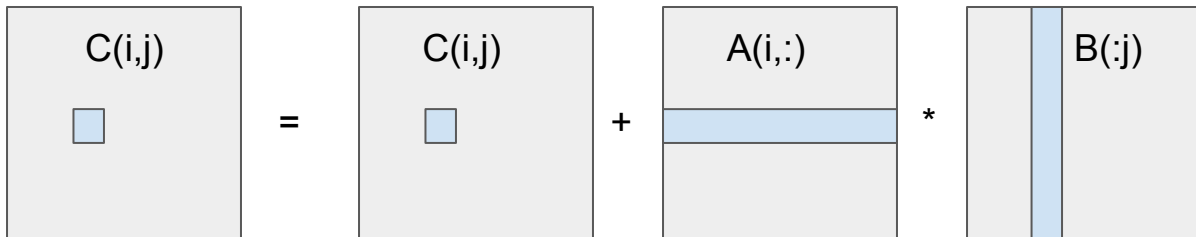
Matrix multiply, basic version: copy optimization

// implements $C = C + A * B$
// A, B, C are NxN matrices

```
for i=1:n    // 1-based indexing
    {read/copy row i of A into cache}
    for j=1:n
        {read/copy C[i,j] into cache}
        {read/copy column j of B into cache}
        for k=1:n
            C[i,j] += A[i,k] * B[k,j]
            {write C[i,j] to slow mem}
```

Some copy optimizations:

Load row of A into cache, reuse in j and k loops
Load column of B into cache, reuse in k loop
Load C[i,j], reuse in k loop, then write after complete



Matrix multiply, basic version: copy optimization

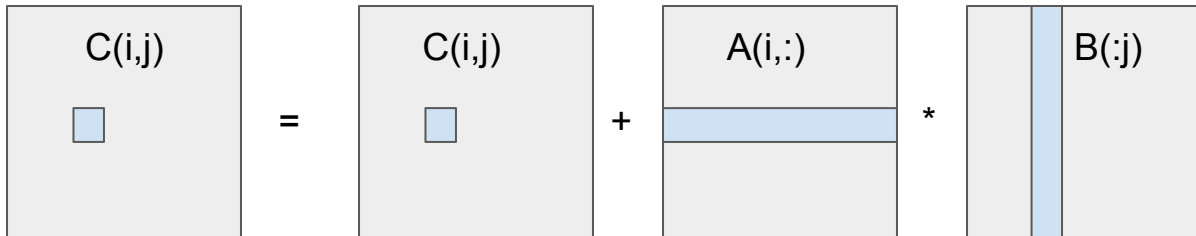
// implements $C = C + A * B$
// A, B, C are $N \times N$ matrices

```
for i=1:n    // 1-based indexing
    {read/copy row i of A into cache}
    for j=1:n
        {read/copy C[i,j] into cache}
        {read/copy column j of B into cache}
        for k=1:n
            C[i,j] += A[i,k] * B[k,j]
            {write C[i,j] to slow mem}
```

of slow memory ops

$m = n^3$ // to read each column of B n^2 times
+ n^2 // to read each row of A once (was N)
+ $2n^2$ // to r+w each element of C once
= $n^3 + 3n^2$

$CI = f/m = 2n^3 / (n^3 + 3n^2) \approx 2$ for large n , so there is no improvement over vector-matrix multiply+copy optimization

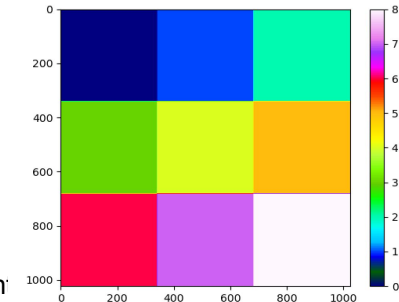
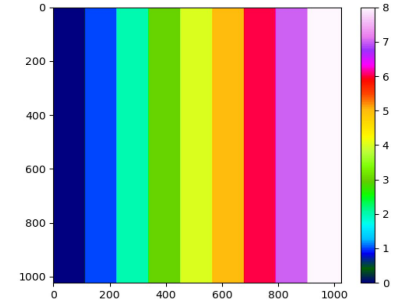
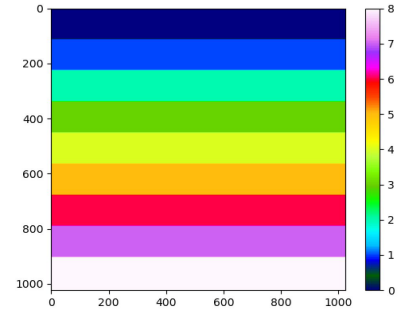


Block/Tile (and copy) optimization

- Copy a subset of a larger problem into a smaller buffer
- Perform operations on the smaller buffer
- The hope: the smaller buffer lives in cache
- The intention: repeated reuse of values in cache lowers AMAT
- Relies on: spatial, temporal locality of accesses

Key difference from copy optimization:

- Rather than copy an entire row-column,
- Copy a “tile” subset
- Thesis: more cache friendly, lowers AMAT



Copyright

Matrix multiply: tiling and blocking optimizations

```
// implements C = C+A*B
// A, B, C are nxn matrices of  $N_b \times N_b$  subblocks
// Assume  $n \% b == 0$ ,  $N_b < n$ 
//  $b = n / N_b$  is called the block size
// each block is of size bxb

for i=1: $N_b$ 
  for j=1: $N_b$ 
    for k=1: $N_b$ 
      C[i,j] += A[i,k] * B[k,j] {mmul on blocks}
```

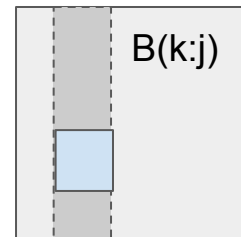
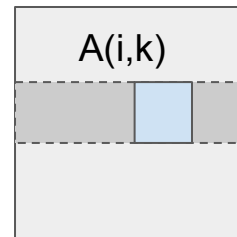
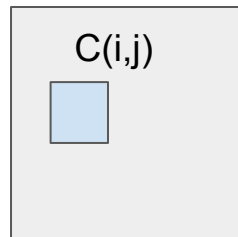
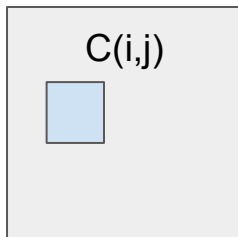
$n \times n$ matrix elements
 $N_b \times N_b$ blocks
Each block is $b \times b$

Key algorithmic ideas:

- Divide larger matrix, $n \times n$, into smaller matrices (blocks) of size $b \times b$
- Perform mmul on $b \times b$ matrices

Why do this?

- What if row of A won't fit in cache?
- Can increase Computational Intensity by performing more FLOPs per memory read
- In other words: runs faster



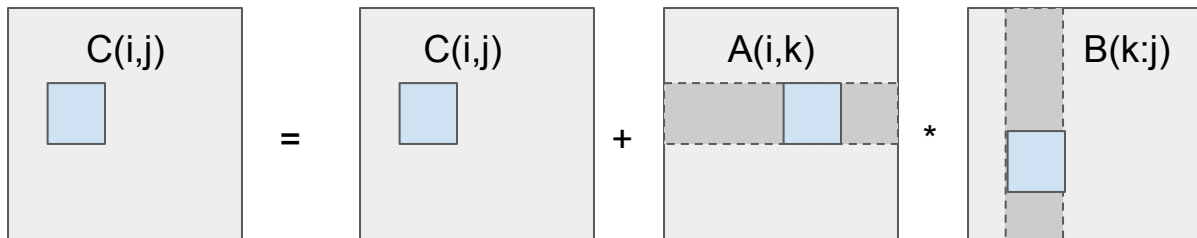
Matrix multiply: tiling and blocking optimizations

```
// implements C = C+A*B  
// A, B, C are nxn matrices of  $N_b \times N_b$  subblocks  
// Assume  $n \% b == 0$ ,  $N_b < n$   
//  $b = n / N_b$  is called the block size  
// each block is of size bxb
```

```
for i=1: $N_b$   
  for j=1: $N_b$   
    for k=1: $N_b$   
       $C[i,j] += A[i,k] * B[k,j]$  {mmul on blocks}
```

3 nested loops inside here
to perform “basic” matrix
multiplication on each bxb
block

nxn matrix elements
 $N_b \times N_b$ blocks
Each block is bxb



Matrix multiply: tiling and blocking optimizations

// A, B, C are $n \times n$ matrices of $N_b \times N_b$ subblocks, each of which is $b \times b$ in size

// $b = n/N_b$, the block size

for $i=1:N_b$

for $j=1:N_b$

{read/copy block $C[i,j]$ into cache}

for $k=1:N_b$

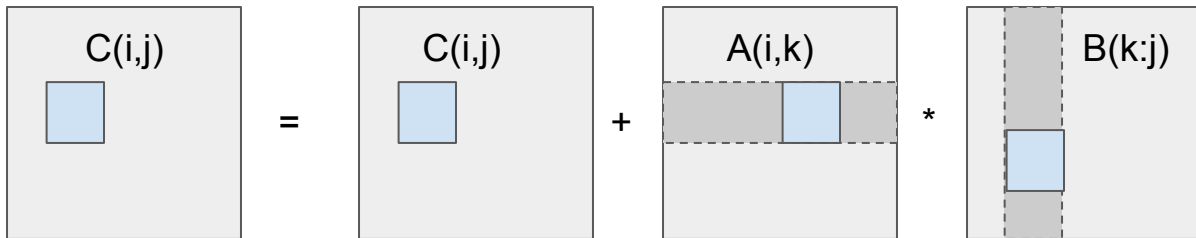
{read/copy block $A[i,k]$ into cache}

{read/copy block $B[k,j]$ into cache}

$C[i,j] += A[i,k] * B[k,j]$ {mmul on blocks}

{write/copy block $C[i,j]$ back to memory}

$n \times n$ matrix elements
 $N_b \times N_b$ blocks
Each block is $b \times b$



Comparison across approaches

Matrix	Basic	Row-copy optimization	2D Blocking/tiling
A	N^3	N^2	$\frac{N^3}{b}$
B	N^3	N^3	$\frac{N^3}{b}$
C	$2N^2$	$2N^2$	$2N^2$
Total (m)	$2N^3 + 2N^2$	$N^3 + 3N^2$	$\frac{2N^3}{b} + 2N^2$



Assumptions

Fast memory was large enough to hold 3 tiles/blocks

Cost of a fast memory access is zero

Memory latency is constant

But, wait! “Copy into fast memory” ???!

There's no such a thing in the C/C++ language standard

→ There's only a single flat memory space

How to copy into “fast memory?”



Array indexing and memory accesses in CP#2 code

Array Indexing

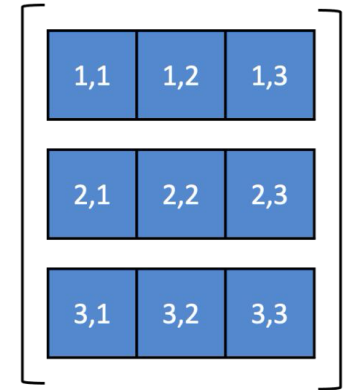
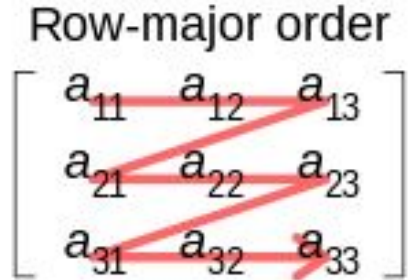
C/C++ use “row major ordering” of arrays in memory

(And C/C++ uses zero-based indexing: the first item in the array is at index=0, the second is at index=1, and so on)

For “structured” MxN arrays:

- The entire MxN array is allocated as one big chunk of memory
- There is some extra bookkeeping involved (by the compiler) to compute row offsets

The situation is different with “ragged” or “unstructured” arrays, where each row may be a different length



Array Indexing

Let's assume A is an array

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}$$

We are interpreting A as a 2D array for a computation, e.g. VMM

But A is actually stored as a 1D array

Question: How do we do logical 2D indexing using only a single indexing variable?

In other words: given two logical indices (row, column), how do we compute a 1D offset into the flat array?

Address	Row-major order
0	a_{11}
1	a_{12}
2	a_{13}
3	a_{21}
4	a_{22}
5	a_{23}

Array Indexing

Example: dimensions of A = 2 rows by 3 columns

We use index=0 (address=0) to access the first item in 1 array, $A_{1,1}$

Row-major order

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}$$

Address	Row-major order
0	a_{11}
1	a_{12}
2	a_{13}
3	a_{21}
4	a_{22}
5	a_{23}

Array Indexing

Example: dimensions of $A = 2$ rows by 3 columns

We use index=0 (address=0) to access the first item in the array, $A_{1,1}$

We use index=1 (address=1) to access the first item in the array, $A_{1,2}$

Row-major order

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}$$

Address	Row-major order
0	a_{11}
1	a_{12}
2	a_{13}
3	a_{21}
4	a_{22}
5	a_{23}

Array Indexing

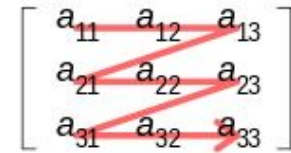
Example: dimensions of A = 2 rows by 3 columns

We use index=0 (address=0) to access the first item in the array, $A_{1,1}$

We use index=1 (address=1) to access the first item in the array, $A_{1,2}$

We use index=2 (address=2) to access the first item in the array, $A_{1,3}$

Row-major order



$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}$$

Address	Row-major order
0	a_{11}
1	a_{12}
2	a_{13}
3	a_{21}
4	a_{22}
5	a_{23}

Array Indexing

Example: dimensions of $A = 2$ rows by 3 columns

We use index=0 (address=0) to access the first item in the array, $A_{1,1}$

We use index=1 (address=1) to access the first item in the array, $A_{1,2}$

We use index=2 (address=2) to access the first item in the array, $A_{1,3}$

What happens when we want to go to the 2nd row in A ?

Row-major order

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}$$

Address	Row-major order
0	a_{11}
1	a_{12}
2	a_{13}
3	a_{21}
4	a_{22}
5	a_{23}

Array Indexing

To access the *second row* in A , we:

- Keep using a 1D index
- But have to skip over the first row of A : 3 items

We use $\text{index}=3$ to access the first item in the second row of the array, $A_{2,1}$

Row-major order

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}$$

Address	Row-major order
0	a_{11}
1	a_{12}
2	a_{13}
3	a_{21}
4	a_{22}
5	a_{23}

Array Indexing

To access the *second row* in A , we:

- Keep using a 1D index
- But have to skip over the first row of A : 3 items

We use index=3 to access the 1st item in the 2nd row of the array, $A_{2,1}$

We use index=4 to access the 2nd item in the 2nd row of the array, $A_{2,2}$

Row-major order

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}$$

Address	Row-major order
0	a_{11}
1	a_{12}
2	a_{13}
3	a_{21}
4	a_{22}
5	a_{23}

Array Indexing

To access the *second row* in A , we:

- Keep using a 1D index
- But have to skip over the first row of A : 3 items

We use index=3 to access the 1st item in the 2nd row of the array, $A_{2,1}$

We use index=4 to access the 2nd item in the 2nd row of the array, $A_{2,2}$

We use index=5 to access the 3rd item in the 2nd row of the array, $A_{2,3}$

Row-major order

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}$$

Address	Row-major order
0	a_{11}
1	a_{12}
2	a_{13}
3	a_{21}
4	a_{22}
5	a_{23}

1D index into a 2D array

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}$$

Let A be an array that is logically M rows by N columns

- Example here: M=2 (2 rows), N=3 (3 columns)

We want to write a doubly nested loop that accesses each item of A using a single 1D index

Adds a value of 1.0 to each item of A[]

Code example

```
for (row=0; row<M; row++)  
    for (col=0; col<N; col++)  
        // do something here with A
```

Compiler Optimization: Loop Interchange

```
// X is a 2D array of size [5000, 100]
// C-style: 5000 rows, 100 columns
// x[i, j] and x[i, j+1] are adjacent memory addresses
```

// before

```
for (int j=0; j<100; j++)
  for (int i=0; i<5000; i++)
    x[i][j] = M_PI * x[i][j];
```

// after

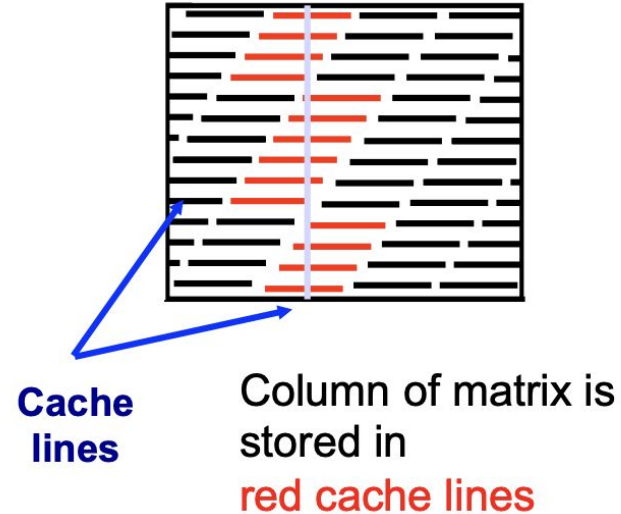
```
for (int i=0; i<5000; i++)
  for (int j=0; j<100; j++)
    x[i][j] = M_PI * x[i][j];
```

Before ordering:

Each inner loop iteration accesses memory in strided (non-contiguous) fashion

After ordering:

Each inner loop iteration accesses contiguous memory addresses.



Optimizing Use of Cache, ctd.

How to Improve Use of Cache?

(Hardware) reduce the hit time: small, simple L1 cache

(Hardware) increase cache bandwidth through pipelined, multi-banked cache

(Hardware) reduce miss penalty (buffer management and priority)

(Software) compiler optimizations and user code: loop reordering, copy optimization, blocking and tiling, in-memory data layout strategies

(Hardware and software) reduce miss penalty rate via parallelism, prefetch

How is data in memory laid out?

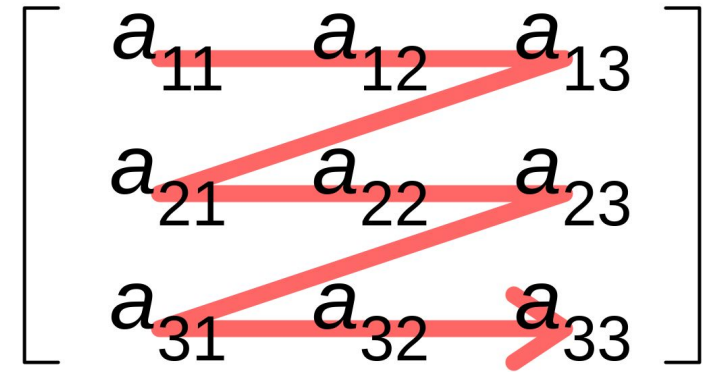
Row-major order:

- Sequential memory addresses within a row of a 2D array (applies to N-D also)
- C, C++, NumPy (Python), ...

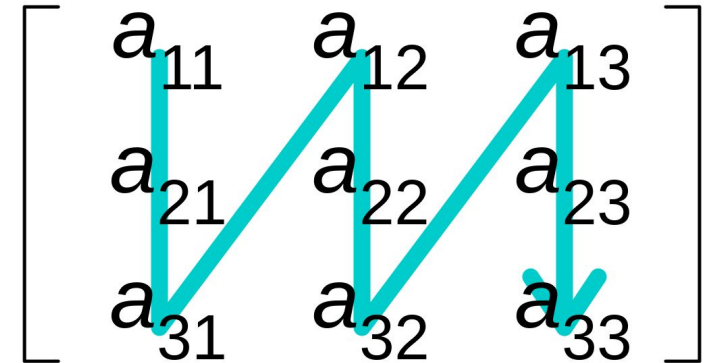
Column-major order:

- Sequential memory addresses within a column of a 2D array
- Fortran (historical significance)

Row-major order



Column-major order



Nearby index, distant address

Regardless of row- or column-major ordering

Nearby in index space may result in large distance in memory address space

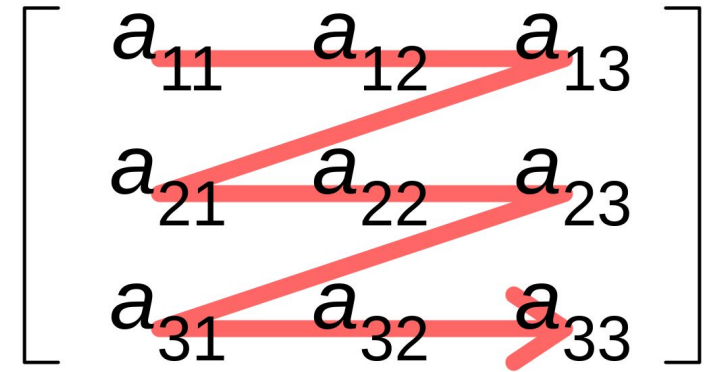
Row-major:

- +/- 1 in the column has high spatial locality
- +/-1 in the row has low spatial locality

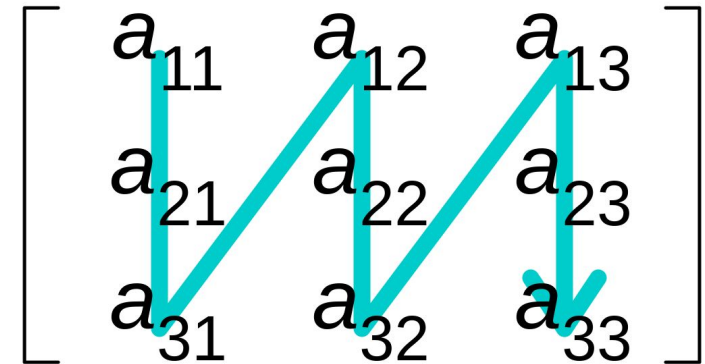
Column-major: the opposite

How can we solve this problem?

Row-major order



Column-major order



Case Study: Do Alternative Memory Layouts Help?

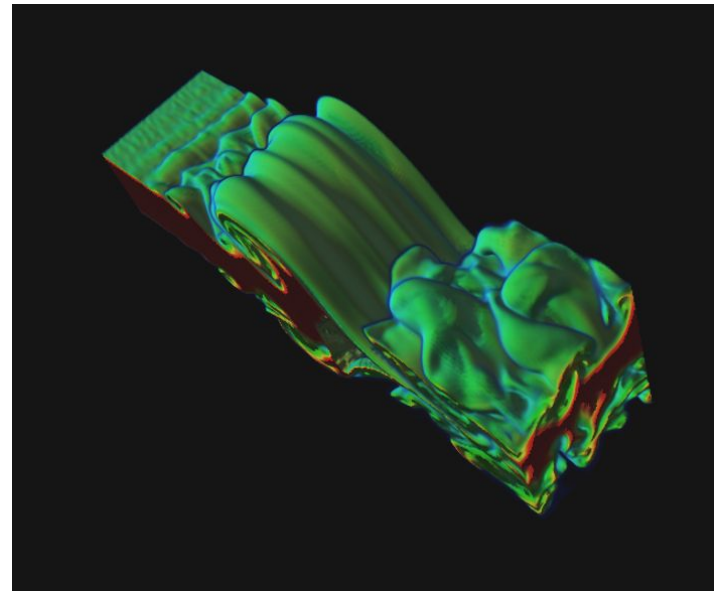
*Improving Performance of Structured-memory,
Data-intensive Applications on Multi-core
Platforms via a Space-filling Curve Memory
Layout*

E. Wes Bethel, David Camp, David Donofrio, Mark
Howison

HPDIC 2015, an IPDPS 2015 Workshop, Hyderabad India, May
2015

PDF of paper: <https://escholarship.org/uc/item/2668z21d>

(separate conference paper presentation, slides not included
here at this time, instead have a look at the PDF of the paper)



The End



A Warner Bros.
PICTURE

